

P1GP03B

Memoria de Practica 1 Matemática Discreta

Miguel Pajuelo Gomez

Jorge Ois de Pascual

Estrategia y decisiones de diseño

En este trabajo decidimos implementar diversos teoremas y demostraciones vistos en clase, complementándolos con algoritmos matemáticos investigados por nuestra cuenta. A medida que avanzábamos con el desarrollo del trabajo, íbamos implementando actualizaciones y cambios en las funciones para optimizar y mejorar la eficiencia de las funciones.

Diseño general de IMatLab

En el `imatlab.py` hemos empezado definiendo patrones de regex para filtrar el input, que en función de los argumentos introducidos al ejecutar el script, puede inicializarse de manera interactiva o Batch.

El Interactivo es en el que el usuario mete los comandos manualmente, si introducen uno no reconocido o con error en el campo, el programa le devuelve un mensaje de error y le sugiere utilizar al comando `--h`, el cual le dará una lista de los comandos aceptados y su funcionamiento.

En el modo Batch, `imatlab` procesa un archivo de entrada y retorna un archivo de salida `fout`, es el método utilizado al ejecutar `imatlab_benchmark`.

Una vez introducido el comando o cargado el archivo (según lo que detecte el sistema mediante regex), cada línea se envía a su función correspondiente definida en `modular.py`. Dependiendo del modo de ejecución, el programa imprimirá la salida directamente en el terminal o la escribirá en un archivo de salida.

En caso de que ocurra un error de cálculo, el programa devuelve el mensaje "NE" (No Evaluado).

Para finalizar hemos decorado el inicio usando arte ASCII para poner un título.

Funciones

Es_Primo(n)

Esta función recibe un entero n y te devuelve si es primo o no.

Para calcularlo hemos usado la el **algoritmo de Miller-Rabin**, un algoritmo probabilístico que usando cierta base es determinista hasta 2^{94} , según lo que descubrieron Sorenson y Webster.

Este algoritmo funciona basándose en el teorema de Fermat, haciendo que un número

primo p y para cualquier base a , $a^{p-1} \equiv 1(p)$. Los compuestos no cumplen esto.

Miller-Rabin factoriza $n - 1$ como $2^s \cdot d$, d siendo impar. Y luego coge un numero de nuestra base, los primeros 15 primos, y calcula $a^d \equiv 1(n)$. Si lo pasa *podría ser primo*, si no lo pasa se empieza a elevar a^s a potencias de 2 hasta llegar a s , si hasta llegar a s no ha salido ninguna congruencia con 1 o -1 entonces es definitivamente compuesto, y se termina la prueba.

Si de *probablemente primo* se hace con el resto de la base hasta que falle, si no falla hay dos casos, si el numero es mayor que 2^{94} entonces en el peor de los casos, que sea un pseudoprimo, tendrá una probabilidad del $\frac{1}{4}^{15}$ o $\frac{1}{2}^{30}$ de ser un falso primo, si es menor que 2^{94} es definitivamente un primo.

Lista_primos(a, b)

Esta función recibe dos números y te dice cuales son los primos entre ellos dos.

Para hacerlo hemos usado el algoritmo de **Criba de Eratóstenes**.

El algoritmo se base en escribir todos los números de 2 a b , b siendo el segundo y superior numero. Empezamos con el 2 y vamos saltando de dos en dos, y cada vez que caiga invalidamos el numero en el que ha caído, es decir todos sus múltiplos, después vamos al siguiente numero no negado, que será el tres, y procedemos a saltar de tres en tres, invalidando los múltiplos de tres, y se sigue con 5,7,11,13,17,19,etc. hasta que se llegue a $\sqrt{b}$ y todos los números no tachados son primos, para hacerlo en un intervalo que no sea $(2,b)$ sino (a,b) se hace normal, y una vez que tengas la lista de 2 a b se quita de la lista todos los valores inferiores a a .

Factorizar(n)

Esta funcion recibe un numero y te devuelve un diccionario de la forma

$[p_1 : a_1, p_2 : a_2, \dots, p_n : a_n]$ con p un primo y a su exponente.

Para esta función hemos hecho nuestro propio algoritmo que se basa en **división sucesiva con optimización** $6k \pm 1$.

Empieza sacando todos los factores pequeños, es decir 2 y3, a partir del 5 va probando divisores p , no prueba todos sino usa la forma $6k \pm 1$, esto es porque todo primo mayor que 3 tiene esa forma y sigue dividiendo hasta que $p^2 > n$, y si queda un numero destino a 1 al final significa que ese numero es primo y lo añade al diccionario.

Mcd(a, b)

Esta función te coge dos números y te devuelve su máximo común divisor.

El algoritmo usado es el **algoritmo de Euclides**.

Este se basa en la propiedad del mcd de $gcd(a, b) = gcd(b, a \bmod b)$, y va ciclando hasta que b sea 0 y por lo tanto la respuesta sea a .

Mcd_n($nlist$)

Esta función recibe una lista y te devuelve el mcd de todos los números.

Para hacerlo hemos usado la función de *functools* *reduce*, una función que recibe otra función y un iterable y va reduciendo el iterable paso a paso, para así aplicar nuestro algoritmo previo de mcd por el **algoritmo de Euclides**.

Bezout(*n*, *m*)

Esta función recibe dos números y te devuelve su mcd y un número tal que

$$\text{mcd}(n, m) = n \cdot x + m \cdot y.$$

Para hacerlo hemos usado el **algoritmo extendido de Euclides**.

Se hacen llamadas recursivas para reducir los números como si fuese el algoritmo simple de Euclides hasta que *m* sea 0, entonces te devolverá los el mcd y *x* e *y*, estos siendo los coeficientes de Bezout.

Coprimos(*n*, *m*)

Esta función recibe dos números y te dice si son coprimos entre si, es decir, que no tienen factores comunes.

Para hacer hemos hecho $\text{mcd}(n, m) == 1$, usando la previamente establecida función de mcd, impulsada por el **algoritmo de Euclides**.

Potencia_mod_p(*base*, *exp*, *p*)

Esta función recibe tres números, la base, el exponente y el módulo *p*, y te devuelve el resto de dividir $\text{base}^{\text{exp}} \pmod{p}$.

Para hacer esto hemos usado **exponenciación binaria**.

Se basa en transformar el exponente en binario y multiplicando y dividiendo cada vez que el dígito sea 1.

Inversa(*n*, *p*)

Esta función recibe dos números, el número que se quiere invertir *n* y el módulo *p*, y devuelve un valor *x* tal que $n \cdot x$ es congruente con 1 $\pmod{p}$.

Para hacerlo, se aplica el algoritmo extendido de Euclides mediante la función

bezout(n, p), que devuelve el máximo común divisor (mcd) de *n* y *p*, junto con los coeficientes de Bézout que cumplen la ecuación $\text{mcd} = n \cdot x + p \cdot y$.

Si el mcd resulta ser 1, significa que *n* y *p* son coprimos, por lo que la inversa modular existe y se obtiene como $x \% p$, asegurando que el resultado esté dentro del rango $[0, p - 1]$.

En caso contrario, cuando el mcd es distinto de 1, la inversa modular no existe

Euler(*n*)

Esta función recibe un número y te calcula su función de phi, es decir, el número de coprimos que tiene *n* de 1 a *n*.

Para hacer esto hemos usado las propiedades de Euler, primero hemos chequeado si *n* = 1 y inicializado una respuesta a 1, después hemos factorizado *n* con nuestra función, y hemos

ciclado el diccionario multiplicando la respuesta por el factor -1 si es un factor único o por $(\text{factor}-1) \cdot \text{factor}^{(\text{expo}-1)}$ si el exponente es mayor que 1. Tras haber ciclado todo el diccionario te devuelve la respuesta.

Legendre(n, p)

Esta función recibe dos números, un entero n y un primo impar p , y devuelve 0, 1 o -1. El numero que devuelve depende de esta ecuación

$$\left(\frac{n}{p}\right) = \begin{cases} 0 & \text{si } n \equiv 0 \pmod{p} \\ 1 & \text{si } n \text{ es un residuo cuadrático mod } p \\ -1 & \text{si } n \text{ no es un residuo cuadrático mod } p \end{cases}$$

Un residuo cuadrático es el resto que deja un cuadrado mod p , ejemplo 3 es residuo cuadrático modulo 22, debido a que $5^2 = 25$ y $25 \equiv 3 \pmod{22}$.

Para hacerlo hemos usado la formula de Euler

$$\left(\frac{n}{p}\right) \equiv n^{\frac{p-1}{2}} \pmod{p}$$

Es decir hemos usado nuestra función de potencia del modo
`res=potencia_mod_p(n,(p-1)/2,p)`

Resolver_sistema_congruencias($alist, blist, plist$)

Esta función recibe tres listas de números: a , los coeficientes de la variable x ; b , los términos independientes; y p , la lista de módulos; de vuelta te devuelve la solución del sistema y el modulo de la solución.

$$\begin{cases} x \equiv a_1 \pmod{p_1} \\ x \equiv a_2 \pmod{p_2} \\ \vdots \\ x \equiv a_n \pmod{p_n} \end{cases}$$

Para hacerlo hemos usado el **Teorema Chino del Resto generalizado**, esto hace que también sea posible con números que no son coprimos entre si, eso si los módulos son compatibles, si no, te devolverá *IncompatibleEquationError*.

Bibliografia

Es_Primo: Miller-Rabin — Wikipedia

https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test

Lista_primos: Sieve of Eratosthenes — Wikipedia

https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes

Factorizar: Integer factorization — Wikipedia

https://en.wikipedia.org/wiki/Integer_factorization

Mcd: Euclidean algorithm — Wikipedia

https://en.wikipedia.org/wiki/Euclidean_algorithm

Mcd_n: Reduce — Libreria functools

<https://docs.python.org/3/library/functools.html>

Bezout: Bézout's identity — Wikipedia

https://en.wikipedia.org/wiki/B%C3%A9zout%27s_identity

Potencia_mod_p: Modular Exponentiation — Wikipedia

https://en.wikipedia.org/wiki/Modular_exponentiation

Inversa: Modular multiplicative inverse — Wikipedia

https://en.wikipedia.org/wiki/Modular_multiplicative_inverse

Legendre: Legendre Symbol — Wikipedia

https://en.wikipedia.org/wiki/Legendre_symbol

Resolver_Sistemas_congruencias: Chinese Remainder Theorem — Wikipedia

https://en.wikipedia.org/wiki/Chinese_remainder_theorem